

# PLANO: Repositório GitHub como Submodule Universal

Data: 2026-08-23

**Objetivo:** Criar repositório reutilizável (fábrica-universal) com APENAS componentes universais, pronto para usar como submodule em qualquer projeto.

**Resultado:** `git submodule add https://github.com/seu-usuario/fabrica-universal.git`  
fábrica-universal → projeto novo pronto com infraestrutura profissional.

## FASE 1: ESTRUTURA DO REPOSITÓRIO UNIVERSAL

### 1.1 – Nome & Descrição

**Nome:** fabrica-universal

**Descrição:** Infrastructure reusável: governança, padrões, economia de tokens, portabilidade multi-IDE – clone como submodule em qualquer projeto

**Repositório:**

`https://github.com/seu-usuario/fabrica-universal`  
(público, MIT license)

### 1.2 – Estrutura de Pastas

|                                      |                           |
|--------------------------------------|---------------------------|
| fabrica-universal/                   |                           |
| ├─ README.md                         | (instruções de uso)       |
| ├─ QUICKSTART.md                     | (5 min setup)             |
| ├─ LICENSE                           | (MIT)                     |
| ├─ .gitignore                        |                           |
| ├─ .claude/                          |                           |
| │   ├─ CLAUDE.md                     | (governança universal)    |
| │   ├─ RTK.md                        | (token economy)           |
| │   └─ settings.json                 | (hooks padrão)            |
| │   └─ skills/                       |                           |
| │       ├─ caveman/SKILL.md          |                           |
| │       ├─ headroom/SKILL.md         |                           |
| │       ├─ lean-ctx/SKILL.md         |                           |
| │       ├─ rtk-memory/SKILL.md       |                           |
| │       └─ pre-flight-check/SKILL.md |                           |
| │   └─ agents/                       | (vazio – será reescrito)  |
| │   └─ commands/                     | (vazio – será reescrito)  |
| │   └─ mcp-servers/                  |                           |
| │       └─ code-review-graph/        | (submodule separado)      |
| ├─ scripts/                          |                           |
| │   ├─ setup-links.ps1               | (portabilidade Win)       |
| │   ├─ setup-links.sh                | (portabilidade Mac/Linux) |
| │   └─ pdf_typst.py                  | (compilação genérica)     |
| │   └─ padroes/                      |                           |
| │       ├─ script-template.py        | (exemplo)                 |
| │       ├─ skill-template.md         | (exemplo)                 |
| │       ├─ command-template.md       | (exemplo)                 |
| │       └─ mcp-template.js           | (exemplo)                 |
| │   └─ validate.py                   | (testar integração)       |
| ├─ templates/                        |                           |
| │   └─ (vazio – será reescrito)      |                           |

|                               |                               |
|-------------------------------|-------------------------------|
| docs/                         |                               |
| ├─ UNIVERSAL-vs-ESPECIFICO.md | (matriz)                      |
| ├─ PADROES.md                 | (skill, script, comando, MCP) |
| ├─ TROUBLESHOOTING.md         | (soluções comuns)             |
| └─ EXEMPLOS.md                | (SaaS, Pesquisa, Consultoria) |
| tests/                        |                               |
| ├─ test-integration.sh        | (validar setup)               |
| ├─ test-syntax.py             | (Python)                      |
| └─ test-junctions.sh          | (links funcionam)             |
| .github/                      |                               |
| ├─ workflows/                 |                               |
| │ └─ validate-submodule.yml   | (CI para submodule)           |
| │ └─ release.yml              | (tag releases)                |
| └─ ISSUE_TEMPLATE/            |                               |
| └─ request-pattern.md         | (solicitar novo padrão)       |

### 1.3 – O Que INCLUIR (Universal = 100%)

✓ **INCLUIR:** - CLAUDE.md (governança R0–R17) - RTK.md (guia token economy) - settings.json (hooks padrão) - 5 skills economia (caveman, headroom, lean-ctx, rtk-memory, pre-flight-check) - setup-links.ps1/sh (portabilidade) - pdf\_typst.py (compilação genérica) - pre-commit hook template - Padrões de script, skill, comando, MCP (templates apenas) - Documentação (universal vs específico, padrões, troubleshooting) - Testes de integração

### 1.4 – O Que EXCLUIR (Específico = 0%)

✗ **EXCLUIR:** - Skills de redação (pesquisador, arquiteto, redator-eita, revisor-tecnico, compilador-\*) - Comandos de criação (/criar-livro, /criar-tcc, /criar-artigo, /compilar-mega-livro) - Tipos de obra (tipos\_obra.py) - Gates de conteúdo (validar-referencias, validar-metricas, validar-escala, etc.) - Templates de output (template\_eita.md, template.typ, template\_lead\_magnet.html) - Scripts específicos (auditar-obra.py, indexar-dossie.py, pool-capitulos.py) - MCPs específicas (db\_state com tabelas de obras/capítulos) - Estrutura de output/ (será criada por novo projeto) - Exemplos de coleção/projeto

## FASE 2: README.md (Instruções Completas)

### # Fábrica Universal – Infraestrutura Reutilizável

> Componentes universais (40% da Fábrica Agêntica) – **\*\*use como submodule em qualquer projeto\*\***.

#### ## 0 Que Você Ganha

- ✓ **\*\*Governança central\*\*** (CLAUDE.md com R1–R17)
- ✓ **\*\*5 Skills de economia de tokens\*\*** (caveman, headroom, lean-ctx, RTK)
- ✓ **\*\*Portabilidade multi-IDE\*\*** (Claude Code, Cursor, OpenCode, Codebuff, VSCode)
- ✓ **\*\*Infraestrutura pronta\*\*** (hooks, CI/CD, auto-memória)
- ✓ **\*\*Padrões reutilizáveis\*\*** (scripts Python, skills, comandos, MCPs)
- ✓ **\*\*Economia severa\*\*** (50–70% menos tokens)

#### ## Setup Rápido (5 min)

#### ### Opção 1: Novo Projeto

```

``bash
# 1. Criar projeto novo
mkdir meu-projeto
cd meu-projeto
git init
git config user.name "Seu Nome"
git config user.email "seu@email.com"

# 2. Adicionar como submodule
git submodule add https://github.com/seu-usuario/fabrica-universal.git fabrica-universal
git submodule update --init --recursive

# 3. Copiar arquivos universais
cp -r fabrica-universal/.claude .
cp -r fabrica-universal/scripts .
cp fabrica-universal/CLAUDE.md .
cp fabrica-universal/.git/hooks/pre-commit .git/hooks/
chmod +x .git/hooks/pre-commit

# 4. Setup portabilidade
./scripts/setup-links.ps1 -Verbose # Windows
# ou
./scripts/setup-links.sh           # Mac/Linux

# 5. Validar
python scripts/validate.py

```

## Opção 2: Projeto Existente

```

cd seu-projeto-existente

# 1. Adicionar submodule
git submodule add https://github.com/seu-usuario/fabrica-universal.git fabrica-universal

# 2. Copiar universal (preserva seu específico)
cp -r fabrica-universal/.claude .
cp -r fabrica-universal/scripts . (cuidado: preservar seus scripts)
cp fabrica-universal/CLAUDE.md .

# 3. Merging com seu CLAUDE.md existente
# (ver seção de merge abaixo)

# 4. Validar
python scripts/validate.py

```

## Estrutura Esperada Pós-Setup

```

seu-projeto/
├── fabrica-universal/           ← Submodule (leitura-só)
│   └── (estrutura do repositório)
├── .claude/                    ← Copiado do submodule
│   ├── CLAUDE.md               (customizar: seu tema)
│   ├── RTK.md
│   ├── settings.json
│   └── skills/{caveman,headroom,...}

```

```

├── scripts/
│   ├── setup-links.ps1 / .sh          (do submodule)
│   ├── pdf_typst.py                  (do submodule)
│   ├── seus-scripts-especificos.py    (seus)
│   └── padroes/                       (templates do submodule)
├── CLAUDE.md                          (hardlink → .claude/CLAUDE.md)
├── .mcp.json                          (seus MCPs)
└── .git/hooks/pre-commit               (do submodule, customizável)

```

## Customizar para Seu Domínio

### Passo 1: CLAUDE.md

```

vim .claude/CLAUDE.md
# Editar:
# - Linha 6: Trocar "Fábrica Agêntica de Publicações" → seu tema
# - Seção 2 (Squad): trocar skills específicas
# - Exemplos: adaptar para seu domínio
# MANTER: Seção 0 (Economia), R1-R17, padrões

```

### Passo 2: Criar Skills de Domínio

```

# Usar template do submodule
cp fabrica-universal/scripts/padroes/skill-template.md .claude/skills/seu-especialista-1/SKILL.md
vim .claude/skills/seu-especialista-1/SKILL.md
# Reescrever lógica 100% (mantém frontmatter + padrão)

```

### Passo 3: Criar Comandos de Domínio

```

# Usar template
cp fabrica-universal/scripts/padroes/command-template.md .claude/commands/seu-comando-1.md
vim .claude/commands/seu-comando-1.md
# Reescrever workflow (mantém fases + checklist)

```

### Passo 4: Gates de Validação

```

# Usar template
cp fabrica-universal/scripts/padroes/script-template.py scripts/validar-seu-criterio.py
vim scripts/validar-seu-criterio.py
# Reescrever lógica (mantém estrutura: console_utf8, JSON, exit code)

```

## Atualizar Submodule

```

# Buscar atualizações
git submodule update --remote

# Mergear mudanças universais novas
git add fabrica-universal
git commit -m "chore(submodule): atualizar fábrica-universal"

# Re-copiar arquivos se houve mudanças
cp -r fabrica-universal/.claude .
./scripts/setup-links.ps1

```

## Troubleshooting

### Q: Submodule não inicializa ao clonar

```
# Ao clonar repository novo:
git clone seu-repo
git submodule update --init --recursive
```

```
# Ou durante clone:
git clone --recurse-submodules seu-repo
```

### Q: Junctions não funcionam

```
# Windows: rodar como Admin
.\scripts\setup-links.ps1 -Verbose
```

```
# Mac/Linux
chmod +x scripts/setup-links.sh
./scripts/setup-links.sh
```

### Q: Validação falha

```
python scripts/validate.py

# Se falhar, rodar testes individuais:
python -m py_compile scripts/*.py
grep "alwaysApply: true" CLAUDE.md
test -f .git/hooks/pre-commit && echo "Hook OK"
```

### Q: CLAUDE.md customizado vs. submodule

```
# Seu CLAUDE.md é cópia (não link)
# Editar livremente sem afetar submodule
vim CLAUDE.md
```

```
# Para trazer atualizações do submodule:
cp fabrica-universal/.claude/CLAUDE.md .claude/CLAUDE.md.upstream
# Fazer merge manual (vim/vimdiff) preservando customizações
```

## Exemplos: Usando em Diferentes Domínios

### SaaS (Desenvolvimento de Features)

```
# 1. Setup universal
git submodule add ... fabrica-universal
cp -r fabrica-universal/.claude .
./scripts/setup-links.ps1
```

```
# 2. Customizar CLAUDE.md
# - Squad: product-manager → architect → dev-senior → qa-engineer
# - Exemplos: /criar-feature em vez de /criar-livro
```

```
# 3. Criar skills SaaS
# - product-manager (visão + requisitos)
# - architect (design + decisões)
# - dev-senior (implementação)
# - qa-engineer (testes ≥90%)
```

```
# 4. Criar comandos SaaS
# - /criar-feature (epic → design → implementação)
# - /debugar-issue (triage → fix → validation)
```

```
# - /refactor-seguro (teste-driven)
# - /deploy-seguro (staging → produção)

# 5. Gates SaaS
# - validar-testes (cobertura ≥90%)
# - validar-performance (P99 < threshold)
# - validar-seguranca (OWASP top 10)
# - validar-compliance (legal/reqs)

# 6. Tipos de artefato
# TIPOS = {"feature": {...}, "refactor": {...}, "hotfix": {...}}
```

## **Pesquisa Acadêmica**

```
# 1. Setup universal (mesmo)

# 2. Customizar CLAUDE.md
# - Squad: literature-researcher → methodologist → statistician → writer
# - Exemplos: /criar-paper em vez de /criar-livro

# 3. Skills acadêmicas
# - literature-researcher (busca + síntese)
# - methodologist (design do estudo)
# - statistician (análise)
# - writer (artigo final)

# 4. Comandos acadêmicos
# - /criar-paper (pesquisa → metodologia → análise → draft)
# - /criar-dissertation (capítulos → revisão → compilação)
# - /criar-poster (resumo visual)

# 5. Gates acadêmicas
# - validar-datasets (integridade, reproducibilidade)
# - validar-metodologia (ética, validade)
# - validar-statistics (p-values, effect sizes)
# - validar-compliance (IRB, regulatório)

# 6. Tipos
# TIPOS = {"paper": {...}, "dissertation": {...}, "poster": {...}}
```

## **Consultoria**

```
# 1. Setup universal

# 2. Customizar CLAUDE.md
# - Squad: estrategista → consultor-senior → designer
# - Exemplos: /criar-proposta em vez de /criar-livro

# 3. Skills
# - estrategista (descoberta + análise)
# - consultor-senior (recomendações)
# - designer (apresentação)

# 4. Comandos
# - /criar-proposta (discovery → análise → recomendações)
# - /criar-relatorio (estrutura → draft → revisão)
# - /criar-playbook (extraction + polish)
```

```
# 5. Gates
# - validar-recomendacoes (viáveis, ROI claro)
# - validar-compliance (legal/regulatório)
# - validar-apresentacao (clarity)

# 6. Tipos
# TIPOS = {"proposta": {...}, "relatorio": {...}, "playbook": {...}}
```

## Contribuindo

Encontrou um padrão melhor? Abra uma issue: - [github.com/seu-usuario/fabrica-universal/issues](https://github.com/seu-usuario/fabrica-universal/issues)

Sugestões: - [ ] Novo padrão (skill, comando, script, MCP) - [ ] Melhoria em documento existente - [ ] Novo exemplo de domínio

## License

MIT — Use livremente em qualquer projeto comercial/pessoal.

## Suporte

- 📖 UNIVERSAL-vs-ESPECIFICO.md — copiar vs reescrever
- 🛠️ PADROES.md — exemplos de cada padrão
- 🚨 TROUBLESHOOTING.md — soluções
- 💡 EXEMPLOS.md — projetos reais (SaaS, Pesquisa, Consultoria)

## FASE 3: QUICKSTART.md (5 Minutos)

```
```markdown
```

```
# QUICKSTART: Setup em 5 Minutos
```

## 1. Clonar com Submodule (1 min)

```
```bash
```

```
git clone --recurse-submodules https://seu-repo-novo.git
cd seu-repo-novo
```

### 2. Copiar Arquivos Universais (1 min)

```
cp -r fabrica-universal/.claude .
```

```
cp fabrica-universal/CLAUDE.md .
```

### 3. Setup Portabilidade (1 min)

```
# Windows
```

```
.\scripts\setup-links.ps1 -Verbose
```

```
# Mac/Linux
```

```
chmod +x scripts/setup-links.sh
```

```
./scripts/setup-links.sh
```

### 4. Validar (1 min)

```
python scripts/validate.py
```

## 5. Customizar (1 min)

# Editar CLAUDE.md (seu tema)

```
vim .claude/CLAUDE.md
```

# Commit

```
git add -A
```

```
git commit -m "chore(init): setup fábrica-universal"
```

**Pronto!** Seu projeto tem infraestrutura profissional. 🚀

## FASE 4: DOCUMENTAÇÃO (docs/)

### 4.1 – UNIVERSAL-vs-ESPECIFICO.md

Conteúdo: matriz de **\*\*copiar 100%\*\*** vs. **\*\*reescrever 100%\*\***  
(usar versão v2 sem tabelas já criada)

### 4.2 – PADROES.md

```markdown

# Padrões Reutilizáveis

## Skill Template

```markdown

name: seu-skill

description: Uma frase

# Skill\_Seu

Você é [especialista].

## Regras

- REGRA 1, 2, 4 do CLAUDE.md

## Passo 1 – [Fase 1]

## Passo 2 – [Fase 2]

## Passo 3 – [Fase 3]

## Checklist

### Script Template

```
#!/usr/bin/env python3
```

```
"""
```

<modulo>: <descricao>

Entrada: <input>

Saída: <output.json>

Garantias: idempotente, sem side effects, exit 0 = sucesso

```
"""
```

```
def console_utf8():
```

```
    for f in (sys.stdout, sys.stderr):
```



```

    try:
        f.reconfigure(encoding="utf-8", errors="replace")
    except:
        pass

def main():
    relatorio = {"status": "falha"}
    try:
        # SUA LÓGICA
        relatorio["status"] = "sucesso"
    finally:
        Path(...).write_text(json.dumps(relatorio))
    return 0 if relatorio["status"] == "sucesso" else 1

```

## Command Template

description: 0 que faz

### ## REQUISITOS CONTRATUAIS

| #   | Requisito | Spec  |
|-----|-----------|-------|
| --- | -----     | ----- |
| R1  | ...       | ...   |

### ## Passo 0 – Preparação

### ## Passo 1 – Fase 1

### ## Passo 2 – Fase 2

### ## Passo 3 – Fase 3

## MCP Template

```

{
  "mcpServers": {
    "seu-mcp": {
      "command": "node",
      "args": ["path/to/server.js"]
    }
  }
}

```

### ### 4.3 – TROUBLESHOOTING.md

Conteúdo: 10–15 problemas + soluções  
(clone de melhorias/2026-08-23-separacao-v2.md seção "Troubleshooting")

### ### 4.4 – EXEMPLOS.md

Conteúdo: Setup passo-a-passo para \*\*SaaS, Pesquisa, Consultoria\*\*

### ## FASE 5: TESTES & CI/CD

### ### 5.1 – tests/test-integration.sh

```
``bash
```

```
#!/bin/bash

# Testar integração após setup

echo "[*] Validando setup..."

# 1. CLAUDE.md
grep "alwaysApply: true" .claude/CLAUDE.md || exit 1
echo "✓ CLAUDE.md OK"

# 2. Skills economia
for skill in caveman headroom lean-ctx rtk-memory pre-flight-check; do
    test -d .claude/skills/$skill || exit 1
done
echo "✓ Skills OK"

# 3. Python syntax
python -m py_compile scripts/*.py || exit 1
echo "✓ Python OK"

# 4. Junctions (Win/Mac/Linux)
test -L agentic || test -d agentic || exit 1
echo "✓ Junctions OK"

# 5. Pre-commit hook
test -f .git/hooks/pre-commit || exit 1
echo "✓ Hook OK"

echo "✅ Setup validado"
exit 0
```

## 5.2 — .github/workflows/validate-submodule.yml

```
name: Validate Submodule

on: [push, pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          submodules: true

      - name: Python syntax
        run: python -m py_compile scripts/*.py

      - name: CLAUDE.md
        run: grep "alwaysApply: true" .claude/CLAUDE.md

      - name: Skills
        run: |
          for skill in caveman headroom lean-ctx rtk-memory pre-flight-check; do
            test -d .claude/skills/$skill || exit 1
          done
```

```
- name: Integration
  run: bash tests/test-integration.sh
```

## FASE 6: RELEASE & MAINTENANCE

### 6.1 — Releases (SemVer)

**v1.0.0 (initial):** -  Governança completa (CLAUDE.md R0–R17) -  5 skills economia -   
Padrões reutilizáveis -  Docs completas

**v1.1.0 (melhorias):** -  Novo padrão (ex.: MCP template) -  Fix em setup-links -  Docs expandidas

### 6.2 — GitHub Releases

```
# Criar tag
git tag -a v1.0.0 -m "Initial release: universal infrastructure"

# Push
git push origin v1.0.0

# Gerar release (GitHub UI)
# Descrição: "Fábrica Universal v1.0.0 – Infraestrutura reusável"
```

### 6.3 — Maintenance

-  GitHub Issues: sugestões de novos padrões
-  Update docs conforme aprendizados
-  Testar em 3+ domínios (SaaS, Pesquisa, Consultoria)
-  Release novo versão a cada melhoria

## FASE 7: MIGRAÇÃO DO PROJETO FABRICA

### 7.1 — Preparar Separação

```
cd proj_fabrica-de-livros

# 1. Criar repositório novo (fabrica-universal)
mkdir ../fabrica-universal
cd ../fabrica-universal
git init
git remote add origin https://github.com/seu-usuario/fabrica-universal.git

# 2. Copiar APENAS universal
cp -r ../proj_fabrica-de-livros/.claude .
cp -r ../proj_fabrica-de-livros/scripts/pdf_typst.py scripts/
cp ../proj_fabrica-de-livros/scripts/setup-links.ps1 scripts/
cp ../proj_fabrica-de-livros/scripts/setup-links.sh scripts/
cp ../proj_fabrica-de-livros/.git/hooks/pre-commit hooks/

# 3. Criar padrões/templates (de projetos existentes)
mkdir -p scripts/padros
# Extrair skill-template.md, script-template.py, command-template.md, mcp-template.js

# 4. Copiar docs
cp -r ../proj_fabrica-de-livros/melhorias/2026-08-23-separacao-universal-especifico-v2.md docs/UNIVERSAL-vs-ESPECIFICO.md
# (adaptar para novo repositório)
```

```
# 5. Criar README.md e QUICKSTART.md
```

```
# 6. Criar tests/
```

```
mkdir -p tests
```

```
# Copiar test-integration.sh
```

```
# 7. Criar .github/workflows/
```

```
# 8. Commit & Push
```

```
git add -A
```

```
git commit -m "chore(init): fábrica-universal v1.0.0"
```

```
git push -u origin main
```

## 7.2 — Atualizar proj\_fabrica como Submodule Consumer

```
cd proj_fabrica-de-livros
```

```
# 1. Adicionar submodule
```

```
git submodule add https://github.com/seu-usuario/fabrica-universal.git fabrica-universal
```

```
# 2. Verificar que .claude/ continua com específico
```

```
# (scripts, skills de redação, commands, etc.)
```

```
# 3. Commit
```

```
git add -A
```

```
git commit -m "refactor: integrar fabrica-universal como submodule"
```

```
# 4. Documentar em README.md
```

```
echo "
```

```
## Estrutura
```

```
- ``fabrica-universal/`` – Submodule com componentes universais (copiar 100%)
```

```
- ``.claude/`` – Customizações para Fábrica de Publicações (reescrever 100%)
```

```
- ``scripts/`` – Scripts específicos de publicação
```

```
" >> README.md
```

## CHECKLIST DE IMPLEMENTAÇÃO

REPOSITÓRIO NOVO (fabrica-universal)

[ ] GitHub repo criado (público, MIT license)

[ ] Estrutura de pastas definida

[ ] README.md completo (8 seções)

[ ] QUICKSTART.md (5 min)

[ ] Docs (universal-vs-especifico, padroes, troubleshooting, exemplos)

[ ] Padrões (skill, script, command, MCP templates)

[ ] Tests (test-integration.sh, test-syntax.py)

[ ] CI/CD workflow (GitHub Actions)

[ ] .gitignore configurado

[ ] LICENSE (MIT)

CONTEÚDO UNIVERSAL

[ ] CLAUDE.md (R0–R17 completos)

[ ] RTK.md (guia token economy)

[ ] settings.json (hooks padrão)

[ ] 5 skills economia (pastas completas)

[ ] setup-links.ps1/.sh (portabilidade)

[ ] pdf\_typst.py (compilação genérica)

[ ] pre-commit hook

#### MIGRANDO proj\_fabrica

- [ ] Adicionar fabrica-universal como submodule
- [ ] Validar que .claude/ específico está preservado
- [ ] Documentar em README.md
- [ ] Commit & Push

#### TESTES

- [ ] Setup em novo projeto (clone + submodule)
- [ ] SaaS (customizar para features)
- [ ] Pesquisa (customizar para papers)
- [ ] Consultoria (customizar para propostas)
- [ ] GitHub Actions passando
- [ ] Documentação completa e clara

### BENEFÍCIOS DESSA ABORDAGEM

| Aspecto               | Antes                                | Depois                            |
|-----------------------|--------------------------------------|-----------------------------------|
| <b>Setup</b>          | Clone tudo, remova 60%,<br>customize | Clone submodule, customize<br>60% |
| <b>Manutenção</b>     | Duplicação de código                 | Uma fonte (submodule)             |
| <b>Atualizações</b>   | Merge manual em cada<br>projeto      | git submodule update --<br>remote |
| <b>Novos projetos</b> | 4–6 semanas                          | 2–3 horas                         |
| <b>Descoberta</b>     | “Como faço?” scattered               | README.md centralizado            |

### PRÓXIMOS PASSOS

1.  **Você tem:** blueprints completos, separação universal-vs-especifico clara
2. → **Criar:** repositório fabrica-universal (usar plano acima)
3. → **Testar:** em 3 domínios diferentes (SaaS, Pesquisa, Consultoria)
4. → **Documentar:** aprendizados + issue templates no repo
5. → **Compartilhar:** com comunidade (comunidad tech, dev forums)

**Resultado:** Você tem uma **infraestrutura reutilizável** que economiza **4–6 semanas** de trabalho em novos projetos. 🚀